CAPSTONE PROJECT 3

Social Media Usage and Emotional Well-Being

Classification Using Machine Learning and Deep Learning

MODEL DEVELOPMENT DOCUMENT

Prepared by: Bhushan Mandekar

Development: Python · Visual Studio Code · Jupyter Notebook · Streamlit

Dataset: Kaggle — Social Media Usage and Emotional Well-Being

October 2026

Document basis: supplied notebook, CSV evaluation exports, JSON metadata and saved figures.

Executive Summary

This capstone develops a multiclass classifier to estimate dominant emotion from demographic and social-media engagement features. The source material documents cleaning of three supplied splits, investigation of duplicate observations and user identifiers, reconstruction of a unique-observation modeling dataset, exploratory analysis, machine-learning comparisons, neural-network training and Streamlit deployment.

The verified modeling dataset contains 261 unique feature-and-target records. A stratified split produced 182 training, 39 validation and 40 test records. Extra Trees achieved the strongest reported validation macro-F1 (0.7497), ahead of the neural network (0.5094). On the held-out test set, the selected Extra Trees model achieved 65.00% accuracy, 0.6169 macro-F1 and 0.6423 weighted F1.

Interpretation must be cautious: only 40 observations are in the final test set; the source dataset exhibited repeated observations and inconsistent user IDs. The resulting demonstration is not a validated clinical or mental-health assessment.

Document Contents

1. Project context and objectives

2. Dataset and data quality

3. Leakage control and split design

4. Exploratory data analysis

5. Feature engineering and preprocessing

6. Machine-learning development

7. Deep-learning development

8. Model selection and final evaluation

9. Prediction and deployment

10. Limitations, recommendations and conclusion

Appendix: Artifact inventory and verification

1. Project Context and Objectives

The central task is multiclass prediction of Dominant_Emotion using eight input variables. The project compares classical supervised-learning algorithms against a feedforward neural network and provides a web interface for a single-record prediction.

The intended outputs are a reproducible training workflow, evaluation artifacts, serialized preprocessing and model files, and a Streamlit interface. Model selection uses validation macro-F1; final test evaluation follows selection.

The target categories recorded in the model metadata are Anger, Anxiety, Boredom, Happiness, Neutral and Sadness. These labels describe the dataset, not verified psychological diagnoses.

Project workflow

- Inspect and repair supplied train/validation/test CSVs.
- Standardize columns, validate numeric and categorical fields, and investigate duplicates.
- Reconstruct a unique-observation modeling dataset and stratify the splits.
- Explore the training data; preprocess without fitting on validation/test records.
- Compare six ML baselines, train a neural network, select by validation macro-F1.
- Evaluate the selected model on the test set and deploy a prediction interface.

2. Dataset and Data Quality

The notebook uses the Kaggle Social Media Usage and Emotional Well-Being dataset. Original supplied files are train.csv, val.csv and test.csv. The raw CSVs required repair and consistency checks before model development.

The notebook documents correction of swapped Age and Gender entries, categorical standardization, numeric conversion, invalid-record screening and duplicate inspection. Duplicate analysis excluded User_ID from the modeling record signature.

Previously observed data-quality diagnostics include 72 User_IDs associated with varying records and 109 unique feature-and-emotion records linked to more than one User_ID. Identical feature combinations did not show contradictory emotion labels in the documented check. These observations make User_ID unsuitable as a straightforward unique observation key.

Field	Role
Age	Input
Gender	Input
Platform	Input
Daily_Usage_Time (minutes)	Input
Posts_Per_Day	Input
Likes_Received_Per_Day	Input
Comments_Received_Per_Day	Input
Messages_Sent_Per_Day	Input
Dominant_Emotion	Target

3. Leakage Control and Split Design

After cleaning, the supplied splits exhibited substantial duplicate overlap. Rather than evaluating on a tiny residual test set after removing all overlapping rows, the workflow combined cleaned records for a deduplicated observation-level modeling set. This changes the original supplied train/test protocol and must be disclosed.

The notebook removed identical feature-plus-target records while excluding User_ID, yielding 261 unique observations. It then created a stratified 70/15/15 split with random_state=42. This procedure aims to prevent exact record repetition across splits, but does not prove that the same underlying individual cannot appear in different splits.

Because the test set was used only after selecting the model on validation performance, its reported scores are an out-of-sample evaluation within this reconstructed split. They remain sensitive to small sample size and the dataset construction.

Partition	Records	Approximate share
Training	182	69.7%
Validation	39	14.9%
Testing	40	15.3%
Total	261	100%

4. Exploratory Data Analysis

Exploratory analysis examines the training data rather than using the held-out test set to make modeling decisions. The notebook includes emotion distribution, age, gender, platform and daily usage visualizations, numerical distributions, and correlation analysis.

The saved figures below are reproduced from the supplied archive. They document what was plotted; causal interpretations are not warranted from these descriptive relationships.

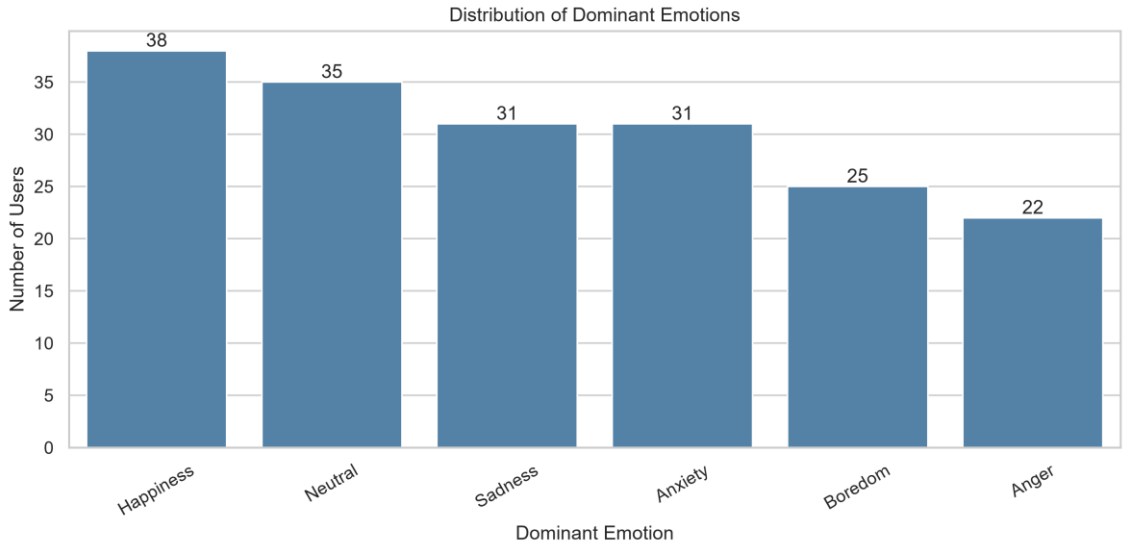


Figure 1. Dominant emotion distribution (supplied project output).

4.1 Numeric Feature Relationships

The correlation matrix summarizes pairwise linear association among numeric input features. It is useful for identifying redundant variables, but cannot by itself establish predictive value or causation.

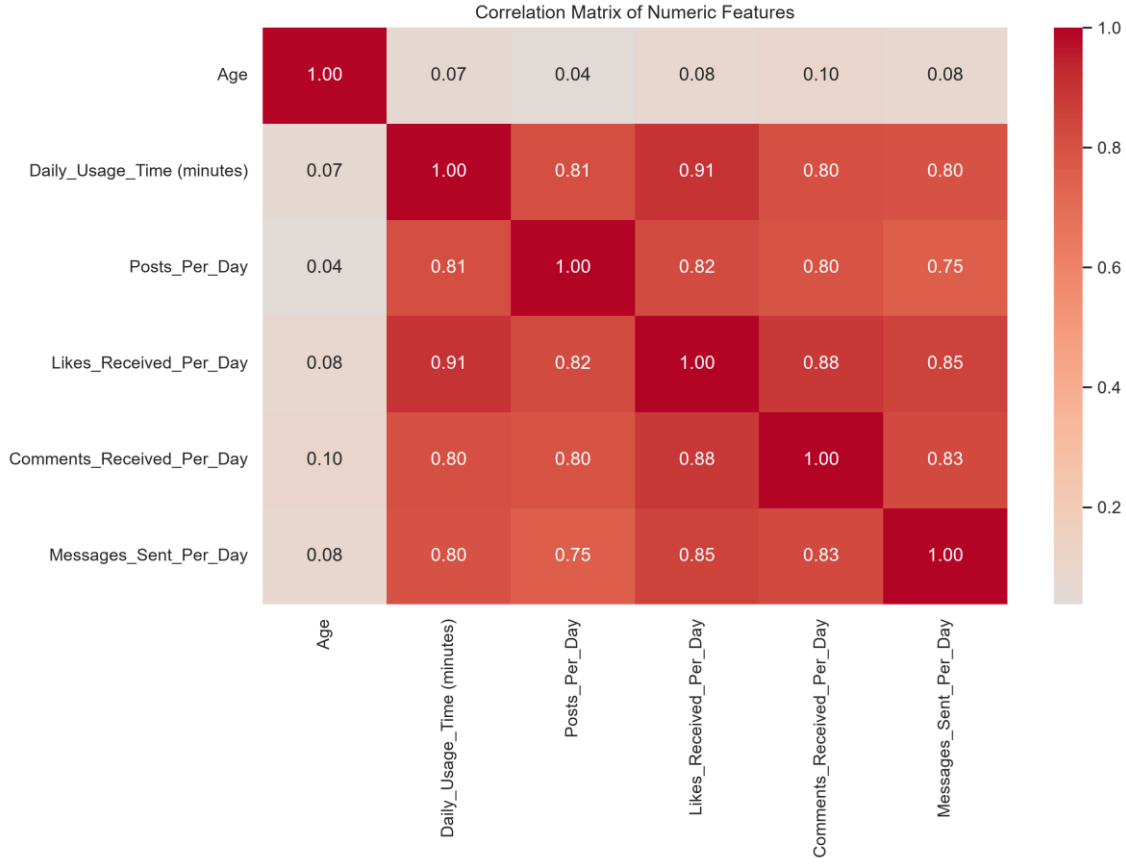


Figure 2. Numeric feature correlation heatmap (supplied project output).

5. Feature Engineering and Preprocessing

The project uses eight input variables: six numeric (Age, Daily_Usage_Time (minutes), Posts_Per_Day, Likes_Received_Per_Day, Comments_Received_Per_Day, Messages_Sent_Per_Day) and two categorical (Gender, Platform).

A ColumnTransformer handles numeric and categorical branches. The numeric branch uses median imputation and StandardScaler; the categorical branch uses most-frequent imputation and OneHotEncoder(handle_unknown="ignore"). The target is converted to integer class labels using LabelEncoder.

The preprocessor is fitted only on training data, then used to transform validation, test and deployment inputs. Serialized artifacts support consistency between model development and the Streamlit application.

Leakage prevention checks

• Fit imputers, scalers and encoders on training data only.

• Use validation data for model comparison, not as an additional training set during the reported comparison.

• Use the held-out test set only after model selection.

• Keep feature order and category handling identical during deployment.

6. Machine-Learning Model Development

The notebook evaluates a majority-class Dummy baseline, Logistic Regression, Random Forest, Extra Trees, Support Vector Machine and K-Nearest Neighbors. All are evaluated on the same validation partition.

Macro-F1 weights each emotion class equally and is therefore appropriate for comparing multiclass classifiers when class-specific performance matters. Weighted metrics also appear in the exports but are affected by class frequency.

Model	Validation accuracy	Validation macro-F1
Extra Trees	0.769	0.750
Random Forest	0.718	0.690
Support Vector Machine	0.590	0.556
Logistic Regression	0.513	0.473
K-Nearest Neighbors	0.436	0.366
Dummy Baseline	0.205	0.057

Extra Trees ranks first by validation macro-F1 (0.7497), followed by Random Forest (0.6896). The Dummy baseline macro-F1 is 0.0567. These figures are from the supplied ml_model_comparison.csv.

6.1 Validation Error Analysis

The confusion matrix displays class-by-class counts for the selected machine-learning model on the validation set. Misclassifications should be interpreted alongside the limited number of observations in each emotion category.

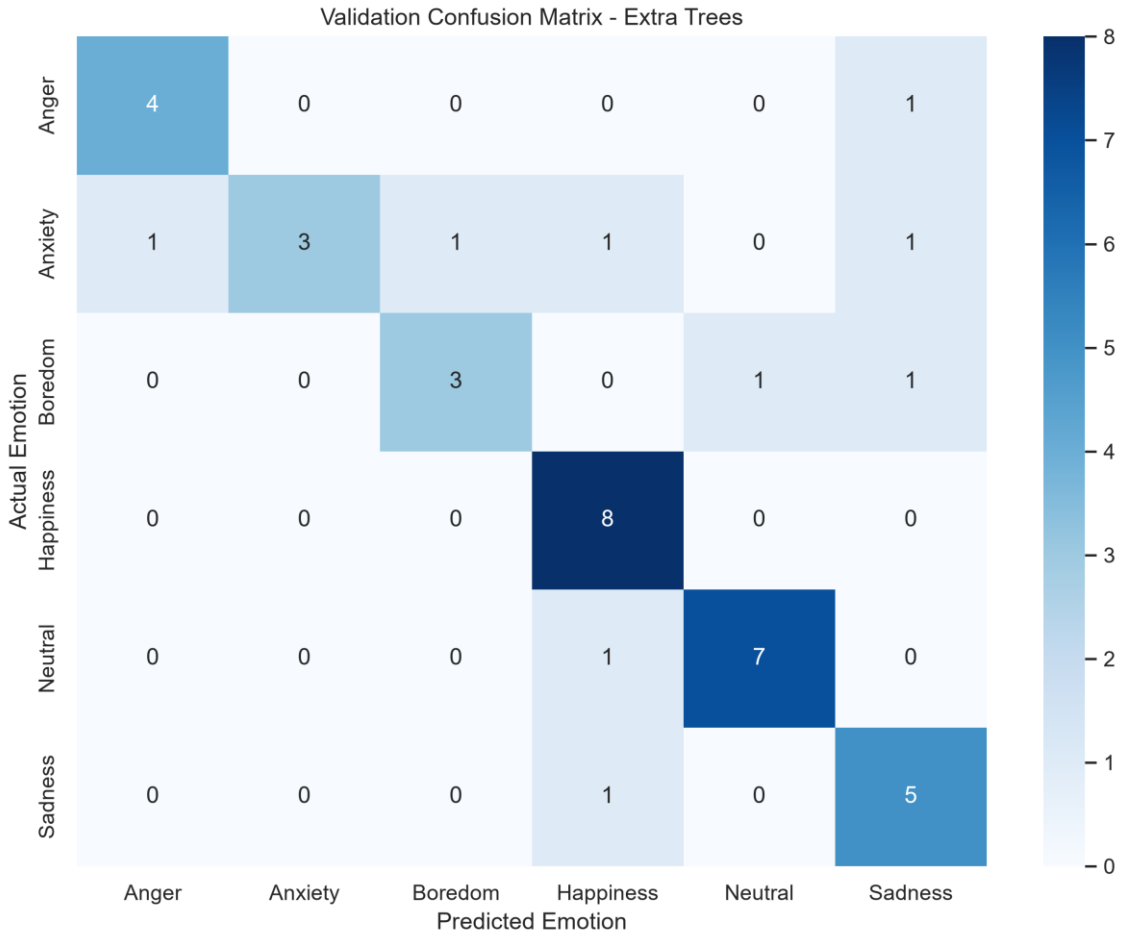


Figure 3. Extra Trees validation confusion matrix (supplied project output).

7. Deep-Learning Model Development

The notebook defines a feedforward neural network for the same preprocessed tabular features. Its architecture uses Dense(64, ReLU), Dropout(0.30), Dense(32, ReLU), Dropout(0.20), and a six-class Softmax output.

Training uses the Adam optimizer, sparse categorical cross-entropy, batch size 16 and a maximum of 100 epochs. EarlyStopping monitors validation loss with patience 10 and restores the best weights; ModelCheckpoint saves the best validation-loss model.

The neural network achieved a reported validation macro-F1 of 0.5094. It did not outperform Extra Trees on the chosen model-selection metric. This outcome is plausible for a small tabular dataset but should not be generalized beyond the evaluated setup.

7.1 Deep-Learning Accuracy Curves

The saved accuracy history shows training and validation accuracy by epoch. Differences between the curves can indicate instability or overfitting, but should be interpreted in the context of a 39-record validation set.

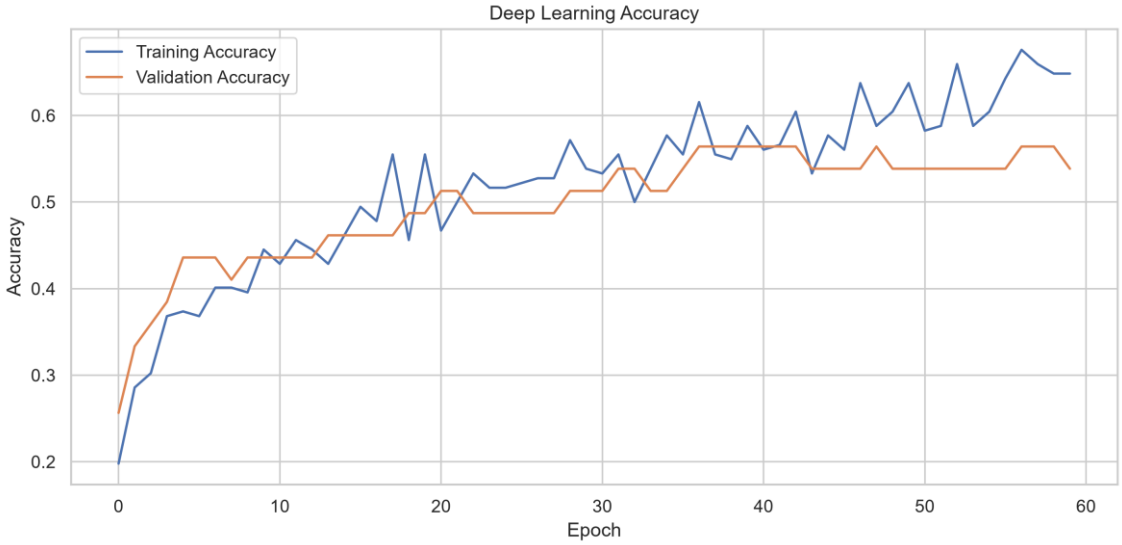


Figure 4. Neural-network training and validation accuracy (supplied project output).

7.2 Deep-Learning Loss Curves

Loss curves offer a complementary view of optimization and generalization. The model checkpoint is selected by validation loss, not by the highest observed training accuracy.

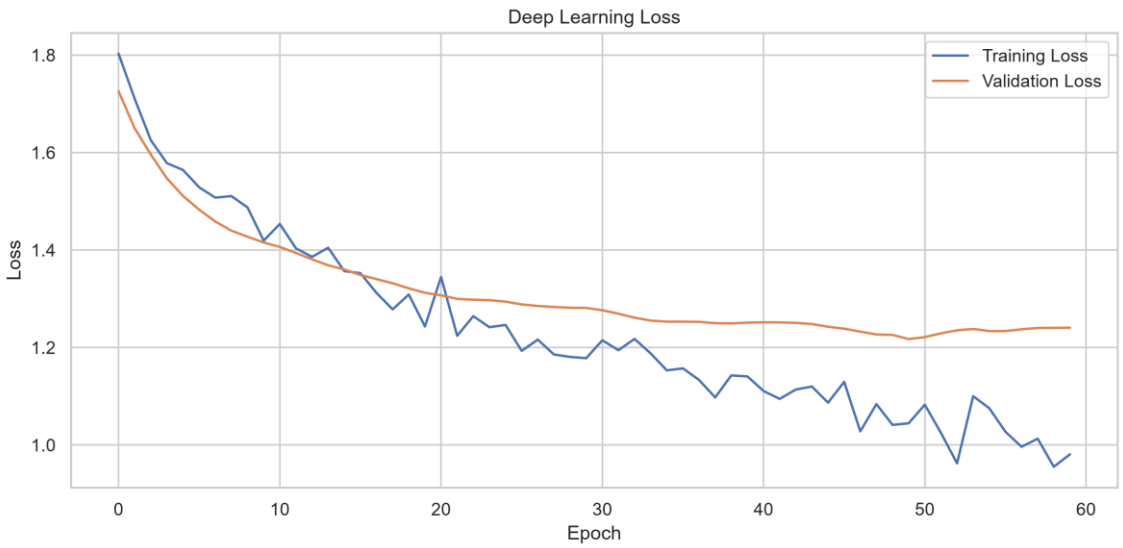


Figure 5. Neural-network training and validation loss (supplied project output).

8. Model Selection and Final Evaluation

Validation macro-F1 is used to select the winning modeling approach. The Extra Trees classifier achieved 0.7497 compared with 0.5094 for the neural network. The selected Extra Trees model was then evaluated on the 40-record held-out test partition.

Test accuracy is 0.6500 (26 correct predictions out of 40). Test macro-F1 is 0.6169 and weighted F1 is 0.6423. The decrease from validation to test macro-F1 highlights the uncertainty of performance estimates on such small samples; it does not by itself establish a specific cause.

Metric	Value
Selected model	Extra Trees
Validation macro-F1	0.7497
Test records	40
Test accuracy	65.00%
Test macro-F1	0.6169
Test weighted F1	0.6423

8.1 Final Test Confusion Matrix

The final test confusion matrix provides class-level evidence behind the aggregate metrics. It should be used alongside the classification report when explaining which emotions are more or less reliably classified.

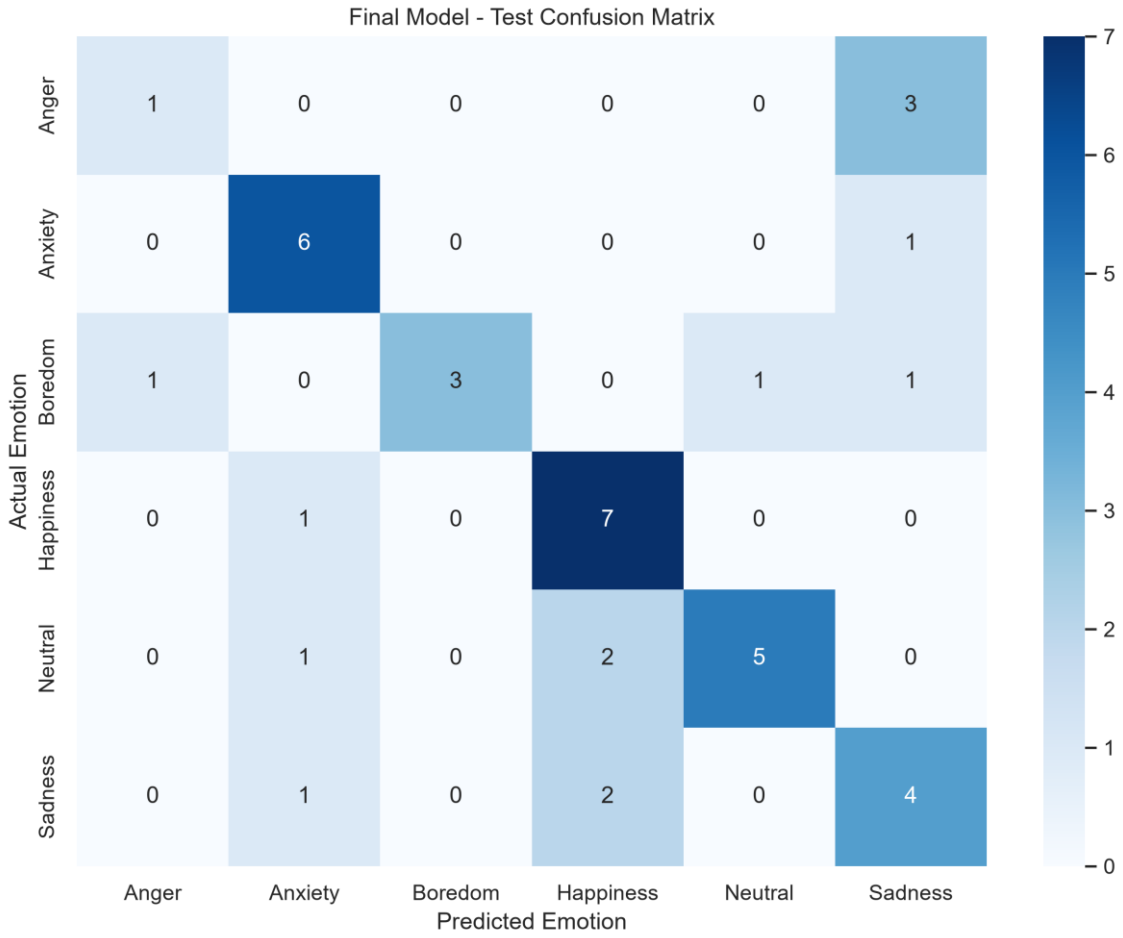


Figure 6. Final Extra Trees test confusion matrix (supplied project output).

9. Prediction and Streamlit Deployment

The final model is serialized for inference alongside the preprocessing transformer and emotion label encoder. model_metadata.json identifies the selected model type as ML, the selected algorithm as Extra Trees, the eight input feature names, the six classes, and the reported final evaluation metrics.

The notebook includes a new-user prediction example and a saved-artifact loading check. The deployment application accepts age, gender, platform and engagement counts, applies the saved transformer, predicts a class, and displays model-estimated probabilities where supported.

The supplied notebook includes a GitHub repository URL and a Streamlit application URL. Their current public accessibility and runtime behavior were not independently verified in preparing this document.

GitHub: https://github.com/Bhushan-Mandekar/social-media-emotion-classification-ml-dl

Streamlit: https://social-media-emotion-classification-ml-dl-xx88jpcjmttv7ivibmx3.streamlit.app/

Deployment artifact contract

Artifact	Role
model_metadata.json	Feature schema and selected model information
preprocessor.pkl	Consistent numeric/categorical transformations
emotion_label_encoder.pkl	Convert predicted class index to emotion label
final_ml_model.pkl	Selected Extra Trees inference model

10. Limitations, Recommendations and Conclusion

The principal methodological limitation is the small deduplicated dataset. With 261 unique observations and 40 held-out test records, class-level scores can change materially with only a few additional correct or incorrect predictions.

The original data exhibited duplicated observations, repeated user identifiers and inconsistent records associated with some identifiers. Removing identical feature-and-label records protects against one form of leakage but cannot establish independence of underlying people or real-world collection processes.

The classification target represents dataset-provided emotion labels; predictions must not be described as diagnoses or reliable measurements of a person's psychological well-being. Probability outputs may be uncalibrated.

Future work should prioritize independently collected observations, documented participant identifiers, external validation, repeated stratified cross-validation within training/model selection, calibration assessment and error analysis for underrepresented classes.

In conclusion, the project demonstrates a coherent end-to-end supervised-learning workflow. The Extra Trees classifier is the best-performing tested approach by validation macro-F1 and achieves 65% test accuracy in the documented reconstructed split. Its strongest value is as an educational prototype, not a production-grade psychological inference system.

Appendix A. Source Artifacts and Verification

The following artifacts were directly inspected from the user-supplied ZIP archive:

- Capstone_3_Emotion_Classification.ipynb
- ml_model_comparison.csv
- ml_vs_dl_comparison.csv
- final_model_evaluation.csv
- project_summary.csv
- dl_training_history.csv
- model_metadata.json
- emotion_distribution.png
- correlation_heatmap.png
- ml_validation_confusion_matrix.png
- dl_accuracy_curve.png
- dl_loss_curve.png
- final_test_confusion_matrix.png
- train_final.csv
- test_final.csv

The source archive did not include a separate validation_final.csv, but the supplied summary and notebook record 39 validation observations. No missing numerical result has been fabricated.

Reproducibility notes

Exact numerical scores are reproduced from exported CSV/JSON artifacts. Method descriptions follow the notebook. Public deployment availability, raw Kaggle provenance and external generalization were not independently verified. To reproduce results, retain the original environment versions and random seed, and check the trained model and preprocessing artifacts against the notebook.

Reference: Kaggle Social Media Usage and Emotional Well-Being dataset — https://www.kaggle.com/datasets/emirhanai/social-media-usage-and-emotional-well-being

Appendix B. Streamlit Application: Implementation Review

This appendix incorporates the separately supplied app.py and requirements.txt. It documents the implementation rather than asserting that the public web service was successfully tested from this report environment. The application URL was provided by the project author.

B.1 Page setup and inference resources

The Streamlit script sets the browser page title to “Social Media Emotion Predictor,” uses a brain emoji icon and a centered layout. It resolves artifact paths relative to the script file using

pathlib.Path(__file__).resolve().parent and a local outputs subdirectory. This design avoids relying on the process working directory.

A function decorated with @st.cache_resource loads the saved JSON metadata, preprocessing transformer, label encoder and selected model. The code chooses final_ml_model.pkl when metadata["model_type"] is ML and final_dl_model.keras when it is DL. The supplied model metadata selects ML, so the Extra Trees inference artifact is the expected deployed model.

B.2 Input collection and validation

UI input	Streamlit control / constraints
Age	number_input; 13–100; default 25
Gender	selectbox: Male, Female, Non-binary
Social media platform	selectbox: Instagram, Facebook, Twitter, LinkedIn, Snapchat, WhatsApp, Telegram, TikTok, YouTube
Daily usage time	number_input; 0–1440 minutes; default 120
Posts per day	number_input; nonnegative; default 3
Likes received per day	number_input; nonnegative; default 50
Comments received per day	number_input; nonnegative; default 10
Messages sent per day	number_input; nonnegative; default 20

The form is submitted with a “Predict Emotion” button. The eight values are assembled into a single-row pandas DataFrame with the exact training-time feature names.

Appendix C. Inference Workflow and Deployment Environment

C.1 End-to-end prediction sequence

1. User completes the Streamlit form and submits the prediction request.

2. The app constructs a one-row DataFrame from the eight fields.

3. The app orders columns according to model_metadata.json and calls preprocessor.transform.

4. For the selected ML model, model.predict returns an encoded class index; predict_proba supplies estimated class probabilities if implemented.

5. emotion_label_encoder.inverse_transform converts the encoded class to a readable emotion label.

6. Streamlit displays the predicted emotion and a bar chart of class probabilities.

The displayed probabilities are explicitly described in the application as model-estimated and potentially uncalibrated. The interface also states that its predictions are for educational and research purposes and are not clinical or psychological assessments.

C.2 Saved model and directory requirements

Relative path	Expected purpose
outputs/model_metadata.json	Selected model type, feature names, classes
outputs/preprocessor.pkl	Training-time transformation pipeline
outputs/emotion_label_encoder.pkl	Encoded-to-readable emotion labels
outputs/final_ml_model.pkl	Extra Trees model selected by current metadata
app.py	Streamlit interface and prediction logic
requirements.txt	Cloud dependency installation list

The app catches loading exceptions, displays an “Unable to load model artifacts” error and stops execution. A cloud deployment therefore requires the outputs directory and its filenames to match exactly, including case and punctuation.

C.3 Dependencies as supplied

The provided requirements.txt lists ipykernel, numpy, pandas, matplotlib, seaborn, scikit-learn, xgboost, tensorflow, imbalanced-learn, joblib, jupyter, streamlit and python-docx. This is the supplied environment declaration; it is not version-pinned. The runtime imports in app.py primarily require streamlit, pandas, numpy, joblib and scikit-learn-compatible model artifacts; TensorFlow is conditionally imported for a DL selection.

Appendix D. Deployment Verification, Risk and Maintenance

D.1 Published application address

Author-provided Streamlit Cloud URL: https://social-media-emotion-classification-ml-dl-xx88jpcjmttv7ivibmx3.streamlit.app/

GitHub repository address recorded in the notebook: https://github.com/Bhushan-Mandekar/social-media-emotion-classification-ml-dl

The report incorporates these addresses as project references. External runtime uptime, GitHub revision consistency and successful live predictions were not independently verified; an attempt to retrieve the Streamlit URL for this update did not provide inspectable application content.

D.2 Suggested acceptance test checklist

- Check that the deployed page loads without an artifact error.
- Submit a plausible example record and confirm an emotion label is shown.
- Confirm the probability chart shows the six metadata-defined classes.
- Check behavior for minimum and maximum accepted numerical inputs.
- Check that the GitHub repository contains the exact outputs/ filenames referenced in app.py.
- Retest after dependency upgrades or model artifact replacements.

D.3 Technical and scientific risks

Risk	Mitigation
Small 40-record test set	Collect more independent observations and report uncertainty
Repeated / ambiguous source records	Track source identifiers and document split provenance
Potentially uncalibrated probabilities	Evaluate calibration before treating probabilities as confidence
Unpinned dependencies	Record working package versions and test clean deployment
Unknown categories / out-of-range data	Monitor inputs and check preprocessing behavior
Mental-health overinterpretation	Retain educational disclaimer; avoid clinical claims

Final assessment: The deployed application is an educational demonstration of a reproducible tabular multiclass prediction workflow. Its measured performance is the reported 65% test accuracy on 40 reconstructed-split test records; this must not be presented as proof of real-world mental-health assessment accuracy.

Appendix E. GitHub Repository and Version Control

E.1 Repository identification

Repository name	social-media-emotion-classification-ml-dl
Repository owner	Bhushan-Mandekar
Repository URL	https://github.com/Bhushan-Mandekar/social-media-emotion-classification-ml-dl
Application URL	https://social-media-emotion-classification-ml-dl-xx88jpcjmttv7ivibmx3.streamlit.app/
Project category	Multiclass classification using machine learning and deep learning
Deployment platform	Streamlit Community Cloud

The repository address and deployment URL were supplied by the project author. The remote repository tree, commit history, branch protection, and current live application state were not independently verified for this document.

E.2 Project layout and deployment artifacts

The following is a recommended, deployment-compatible structure based on the provided app.py and project artifacts; it is not an independently verified snapshot of the GitHub repository:

- app.py — Streamlit prediction interface and inference workflow
- requirements.txt — Python package dependencies for cloud installation
- Capstone_3_Emotion_Classification.ipynb — development, EDA, training and evaluation
- outputs/model_metadata.json — selected model type, feature order and class information
- outputs/preprocessor.pkl — fitted preprocessing pipeline
- outputs/emotion_label_encoder.pkl — class label encoder
- outputs/final_ml_model.pkl — final model file required when metadata model_type is ML
- outputs/final_dl_model.keras — required instead when metadata model_type is DL
- README.md — project overview, installation, usage and reproducibility instructions
- data/ or exported CSVs — only if sharing is permitted and file sizes are appropriate

Path names are case-sensitive on the Linux-based Streamlit deployment environment. The directory must be named exactly outputs, without Markdown backticks or extra spaces. The final model filename must agree with model_metadata.json and the app.py loading branch.

E.3 Version-control and reproducibility recommendations

- Maintain a concise README with project objective, dataset source, workflow, key results, environment setup, usage instructions and both public URLs.
- Use .gitignore to exclude virtual environments, local caches, credentials and unnecessary temporary outputs. Retain essential inference artifacts in the deployment repository or a supported artifact store.
- Pin tested package versions where practical and rebuild the environment after dependency updates.
- Do not commit API keys, passwords, tokens or personally identifying records.
- Record the tested Git commit hash and deployment date before final submission.
- Confirm that a fresh installation can start the application and reproduce a sample prediction.

Appendix F. Project Completion and Submission Checklist

The checklist distinguishes deliverables documented by supplied project files from verification steps that must be performed by the project author before submission.

Item	Evidence	Final action
Jupyter notebook	Supplied notebook	Re-run key cells and retain final outputs
Model evaluation	Supplied CSV/JSON results	Check that reported metrics match artifacts
EDA and training charts	Supplied image files	Confirm captions and axes are readable
Streamlit application	Supplied app.py	Perform live sample prediction
Dependencies	Supplied requirements.txt	Verify clean deployment installation
GitHub repository	Author-provided URL	Verify README, file paths and accessible branch
Model files	Referenced by app.py	Confirm selected artifact exists in outputs/
Deployment	Author-provided URL	Check current app loads and predicts
Project documentation	This MDD	Review formatting and update submission metadata
Ethics and limitations	Discussed in MDD	Avoid clinical claims; disclose small test sample

F.1 Final presentation and demonstration

For a project review, demonstrate the deployed input form, one prediction, the probability chart, the final model selection rationale, and the comparison of validation and test performance. Explain that model-estimated probabilities may not be calibrated and that the system is not a psychological diagnostic tool.

F.2 Claims requiring final author verification

- GitHub repository visibility, latest commit and exact remote folder/file names.
- Whether the live Streamlit app currently loads successfully and returns predictions.
- Python/package versions actually used in the final deployed build.
- Consistency of deployed model artifacts with the evaluation metrics documented in the notebook.

Submission readiness: The report documents the supplied evidence and provides a reproducibility-oriented checklist. Completion of the live verification items above should be recorded before the final academic submission.